

SQL

Εργαστήριο 2

Εισαγωγή στη SQL (Structured Query Language)

- ❑ Η “Γλώσσα” των βάσεων δεδομένων
- ❑ Structured Query Language (SQL)
- ❑ Τυποποιημένη γλώσσα (ISO, ANSI).
- ❑ Χρησιμοποιείται για διαχείριση και ανάκτηση δεδομένων σε σχεσιακές βάσεις δεδομένων.

Εισαγωγή στη SQL

(Structured Query Language)

Έτος	Όνομα Προτύπου	Περιγραφή
1986	SQL-86 (SQL-87 ANSI)	Πρώτη έκδοση, βασική σύνταξη SQL
1989	SQL-89 (SQL1)	Μικρές επεκτάσεις
1992	SQL-92 (SQL2)	Σημαντική αναβάθμιση — βάση των περισσότερων SQL DBs
1999	SQL:1999 (SQL3)	Προσθήκη αντικειμενοστραφών δυνατοτήτων, triggers, recursion

Σχεσιακές βάσεις δεδομένων

- ❑ Βάσεις που οργανώνουν τα δεδομένα σε **πίνακες** (rows & columns).
- ❑ Σχέσεις μεταξύ πινάκων μέσω **πρωτεύοντων** και **ξένων κλειδιών**.
- ❑ Βασίζονται στο **σχεσιακό μοντέλο** του Codd (1970).

Σχεσιακές βάσεις δεδομένων

Παραδείγματα:

- ☐ MySQL
- ☐ PostgreSQL
- ☐ Oracle Database
- ☐ Microsoft SQL Server

Πλεονεκτήματα:

- ☐ Οργάνωση και συνέπεια δεδομένων
- ☐ Ευκολία στην ανάκτηση με SQL
- ☐ Ακεραιότητα μέσω περιορισμών (constraints)

Άλλα είδη βάσεων δεδομένων

- ❑ **Hierarchical:** Δομή δέντρου, κάθε εγγραφή έχει έναν γονέα (π.χ. IBM IMS).
- ❑ **Network:** Επιτρέπουν πολλαπλές σχέσεις μεταξύ εγγραφών (π.χ. CODASYL).
- ❑ **Object-Oriented:** Βασισμένες σε αντικείμενα – υποστηρίζουν κληρονομικότητα, μεθόδους.
- ❑ **NoSQL (Not Only SQL):** Για ευέλικτα ή μεγάλα δεδομένα – παραδείγματα:
 - ❑ Document-based (MongoDB)
 - ❑ Key-Value (Redis)
 - ❑ Columnar (Cassandra)
 - ❑ Graph (Neo4j)
- ❑ **Graph:** Χρήση κόμβων και συνδέσεων, κατάλληλες για κοινωνικά δίκτυα.

Σχεσιακές βάσεις δεδομένων

- ☐ Δημιουργία πινάκων και βάσεων δεδομένων
- ☐ Εισαγωγή, τροποποίηση, διαγραφή δεδομένων
- ☐ Ερωτήματα (queries) για ανάκτηση πληροφοριών
- ☐ Έλεγχος πρόσβασης & ασφάλεια

Σχεσιακές βάσεις δεδομένων

Κύρια χαρακτηριστικά:

- ❑ **Πίνακες (relations):** Κάθε πίνακας περιέχει δεδομένα για μία οντότητα (π.χ. πελάτες, προϊόντα).
- ❑ **Γραμμές (rows):** Κάθε γραμμή είναι μια εγγραφή (instance) της οντότητας.
- ❑ **Στήλες (columns):** Κάθε στήλη είναι ένα χαρακτηριστικό (π.χ. όνομα, τιμή).
- ❑ **Πρωτεύον κλειδί (Primary Key):** Μοναδικό αναγνωριστικό για κάθε γραμμή.
- ❑ **Ξένο κλειδί (Foreign Key):** Δημιουργεί **σύνδεση** με πίνακα που περιέχει σχετική πληροφορία.

Βασικές εντολές SQL

- ☐ SELECT – Ανάκτηση δεδομένων
- ☐ INSERT – Εισαγωγή νέων δεδομένων
- ☐ UPDATE – Τροποποίηση υπαρχόντων δεδομένων
- ☐ DELETE – Διαγραφή δεδομένων
- ☐ CREATE – Δημιουργία πινάκων/βάσεων
- ☐ DROP – Διαγραφή πινάκων/βάσεων

Γενική δομή SELECT εντολής:

Τμήμα	Λειτουργία
SELECT	Ποιες στήλες να εμφανιστούν (π.χ. SELECT όνομα, ηλικία)
FROM	Από ποιον πίνακα (ή πίνακες)
WHERE	Φιλτράρισμα εγγραφών (π.χ. WHERE ηλικία > 30)
GROUP BY	Ομαδοποίηση αποτελεσμάτων
HAVING	Φιλτράρισμα ομάδων (μετά το GROUP BY)
ORDER BY	Ταξινόμηση αποτελεσμάτων

Τελεστές στην SQL

☐ Αριθμητικοί τελεστές

Τελεστής	Περιγραφή	Παράδειγμα
+	Πρόσθεση	SELECT 2 + 3
-	Αφαίρεση	SELECT 5 - 2
*	Πολλαπλασιασμός	SELECT 4 * 6
/	Διαίρεση	SELECT 10 / 2
%	Υπόλοιπο διαίρεσης	SELECT 7 % 3

Τελεστές στην SQL

❑ Τελεστές σύγκρισης

Τελεστής	Περιγραφή	Παράδειγμα
=	Ίσο	WHERE ηλικία = 25
<> ή !=	Διαφορετικό	WHERE φύλο != 'Α'
<	Μικρότερο	WHERE τιμή < 100
>	Μεγαλύτερο	WHERE ηλικία > 18
<=	Μικρότερο ή ίσο	WHERE τιμή <= 50
>=	Μεγαλύτερο ή ίσο	WHERE ηλικία >= 21

Τελεστές στην SQL

☐ Λογικοί τελεστές

Τελεστής	Περιγραφή	Παράδειγμα
AND	Και	WHERE ηλικία > 18 AND φύλο = 'Α'
OR	Ή	WHERE πόλη = 'Αθήνα' OR πόλη = 'Πάτρα'
NOT	Όχι (άρνηση)	WHERE NOT (πόλη = 'Αθήνα')

Τελεστές στην SQL

❑ Ειδικοί τελεστές

Τελεστής	Περιγραφή	Παράδειγμα
BETWEEN	Εύρος	WHERE ηλικία BETWEEN 18 AND 30
IN	Ανήκει σε λίστα	WHERE πόλη IN ('Αθήνα', 'Θεσσαλονίκη')
LIKE	Αντιστοίχιση προτύπου (pattern)	WHERE όνομα LIKE 'Μαρ%'
IS NULL	Έλεγχος για NULL τιμές	WHERE διεύθυνση IS NULL

Τελεστές στην SQL

❑ Συνολοθεωρητικοί τελεστές

Τελεστής	Περιγραφή	Κρατά διπλότυπα;	Παράδειγμα Χρήσης	Αποτέλεσμα
UNION	Ενώνει τα αποτελέσματα από δύο ερωτήματα	Όχι	Φοιτητές σε Μαθηματικά ή Φυσική	Όλα τα ονόματα, χωρίς επαναλήψεις
UNION ALL	Όπως το UNION, αλλά κρατά τα διπλότυπα	Ναι	Φοιτητές σε Μαθηματικά ή Φυσική	Όλα τα ονόματα, με επαναλήψεις
INTERSECT	Επιστρέφει τις κοινές εγγραφές μεταξύ δύο ερωτημάτων	Όχι	Φοιτητές και στα δύο μαθήματα	Μόνο όσοι είναι και στα δύο
EXCEPT	Επιστρέφει τις εγγραφές του πρώτου ερωτήματος που δεν είναι στο δεύτερο	Όχι	Φοιτητές στα Μαθηματικά όχι στη Φυσική	Όσοι είναι μόνο στα Μαθηματικά

Τελεστές στην SQL

❑ Συναθροιστικές συναρτήσεις

Συνάρτηση	Περιγραφή	Επιστρεφόμενο Τύπος	Παράδειγμα Χρήσης	Τι Επιστρέφει
COUNT(*)	Μετρά όλες τις γραμμές	INTEGER	SELECT COUNT(*) FROM students;	Πλήθος όλων των γραμμών
COUNT(col)	Μετρά τις μη-NULL τιμές της στήλης col	INTEGER	SELECT COUNT(grade) FROM grades;	Πλήθος των βαθμών που δεν είναι NULL
SUM(col)	Υπολογίζει το άθροισμα των τιμών της στήλης	Αριθμητικός	SELECT SUM(salary) FROM employees;	Σύνολο των μισθών
AVG(col)	Υπολογίζει τον μέσο όρο	Αριθμητικός	SELECT AVG(score) FROM exams;	Μέσος όρος βαθμών
MIN(col)	Επιστρέφει τη μικρότερη τιμή	Όποιος τύπος έχει η στήλη	SELECT MIN(age) FROM students;	Τη μικρότερη ηλικία
MAX(col)	Επιστρέφει τη μεγαλύτερη τιμή	Όποιος τύπος έχει η στήλη	SELECT MAX(salary) FROM employees;	Τον μεγαλύτερο μισθό

Βασικές εντολές SQL

Τελεστές ALL και ANY:

- ❑ **ALL:** Η συνθήκη πρέπει να ισχύει για **όλες** τις τιμές που επιστρέφει το υποερώτημα.
- ❑ **ANY (ή SOME):** Η συνθήκη αρκεί να ισχύει για **τουλάχιστον μία** τιμή από το υποερώτημα.

Βασικές εντολές SQL

Πωλήσεις

ποσό
100
200
300

Πελάτες

όνομα	ποσό_αγοράς
Μαρία	250
Νίκος	90

- ☐ Ποιοί είναι οι πελάτες των οποίων το ποσό_αγοράς είναι **μεγαλύτερο από όλα** τα ποσά πωλήσεων.

SELECT όνομα

FROM πελάτες

WHERE ποσό_αγοράς > **ALL** (*SELECT* ποσό *FROM* πωλήσεις);

- ☐ ποσό_αγοράς > 100 AND ποσό_αγοράς > 200 AND ποσό_αγοράς > 300
- ☐ Άρα κανένα αποτέλεσμα!

Βασικές εντολές SQL

Πωλήσεις

ποσό
100
200
300

Πελάτες

όνομα	ποσό_αγοράς
Μαρία	250
Νίκος	90

- ☐ Ποιοί είναι οι πελάτες με ποσό_αγοράς **μεγαλύτερο από τουλάχιστον ένα** ποσό από τις πωλήσεις.

SELECT όνομα

FROM πελάτες

*WHERE ποσό_αγοράς > **ANY** (SELECT ποσό FROM πωλήσεις);*

- ☐ *ποσό_αγοράς > 100 OR ποσό_αγοράς > 200 OR ποσό_αγοράς > 300*
- ☐ *Για τη Μαρία τουλάχιστον μία συνθήκη είναι αληθής, άρα συμπεριλαμβάνεται στο αποτέλεσμα.*

Βασικές εντολές SQL

Τελεστές EXISTS / NOT EXISTS

- ❑ **EXISTS**: Επιστρέφει true αν το υποερώτημα επιστρέφει τουλάχιστον μία γραμμή.
- ❑ **NOT EXISTS**: Επιστρέφει true αν το υποερώτημα δεν επιστρέφει καμία γραμμή

Βασικές εντολές SQL

Πελάτες

id	όνομα
1	Μαρία
2	Νίκος

Παραγγελίες

πελάτης_id	ποσό
1	100
1	50

- ❑ Ποιοί πελάτες έχουν κάνει τουλάχιστον μία παραγγελία;

```
SELECT όνομα  
FROM πελάτες p  
WHERE EXISTS (  
  SELECT pr.πελάτης_id  
  FROM παραγγελίες pr  
  WHERE pr.πελάτης_id = p.id  
);
```

- ❑ Μόνο η Μαρία, γιατί ο Νίκος δεν έχει παραγγελίες.

Βασικές εντολές SQL

Πελάτες

id	όνομα
1	Μαρία
2	Νίκος

Παραγγελίες

πελάτης_id	ποσό
1	100
1	50

Ποιοί πελάτες δεν έχουν κάνει καμία παραγγελία;

```
SELECT όνομα  
FROM πελάτες p  
WHERE NOT EXISTS (  
  SELECT pr.πελάτης_id  
  FROM παραγγελίες pr  
  WHERE pr.πελάτης_id = p.id  
);
```

❑ Μόνο ο Νίκος.

Χρήση του διπλού NOT EXIST

- ❑ Βρείτε τους φοιτητές που είναι εγγεγραμμένοι σε όλα τα μαθήματα του Καθηγητή Χ;
- ❑ ή
- ❑ Βρείτε τους φοιτητές για τους οποίους δεν υπάρχει κάποιο μάθημα που διδάσκει ο Καθηγητής Χ στο οποίο δεν είναι εγγεγραμμένοι.

students

student_id	student_name
1	Άννα
2	Κώστας
3	Νίκη

courses

course_id	title	professor
101	Μαθηματικά	Καθηγητής Χ
102	Φυσική	Καθηγητής Χ
103	Χημεία	Καθηγητής Υ

enrollments

student_id	course_id
1	101
1	102
2	101
3	102

Χρήση του διπλού NOT EXISTS

```
SELECT s.student_id, s.student_name
FROM students s
WHERE NOT EXISTS (
  SELECT c.course_id
  FROM courses c
  WHERE c.professor = 'Καθηγητής X'
  AND NOT EXISTS (
    SELECT e.course_id
    FROM enrollments e
    WHERE e.student_id = s.student_id
    AND e.course_id = c.course_id
  )
);
```

- ☐ Για κάθε φοιτητή, ψάχνει όλα τα μαθήματα του Καθηγητή X
- ☐ Για κάθε τέτοιο μάθημα, ελέγχει αν δεν είναι γραμμένος σε αυτό (εσωτερικό NOT EXISTS)
- ☐ Αν υπάρχει έστω ένα τέτοιο μάθημα, τότε το εξωτερικό NOT EXISTS αποτυγχάνει — δηλαδή ο φοιτητής αποκλείεται.
- ☐ Μόνο αν είναι γραμμένος σε όλα, περνάει!

Χρήση του διπλού NOT EXIST

```
SELECT s.student_id, s.student_name  
FROM students s
```

- ☐ Επιλέγουμε όλους τους φοιτητές από τον πίνακα students
- ☐ Θέλουμε τελικά μόνο εκείνους που πληρούν την προϋπόθεση στο WHERE

WHERE NOT EXISTS (

Εδώ ξεκινάει η **λογική του ελέγχου**:

- ☐ Επιλέγουμε μόνο τους φοιτητές για τους οποίους **ΔΕΝ ΙΣΧΥΕΙ** κάτι συγκεκριμένο (το εσωτερικό ερώτημα)

Χρήση του διπλού NOT EXISTS

```
SELECT c.course_id
FROM courses c
WHERE c.professor = 'Καθηγητής X'
AND NOT EXISTS (
```

- ❑ Εξετάζουμε όλα τα μαθήματα του Καθηγητή X
- ❑ Και για κάθε ένα από αυτά, βλέπουμε αν ΔΕΝ είναι εγγεγραμμένος ο τρέχων φοιτητής.

```
SELECT e.course_id
FROM enrollments e
WHERE e.student_id = s.student_id
AND e.course_id = c.course_id
```

- ❑ Ελέγχουμε αν υπάρχει εγγραφή του τρέχοντος φοιτητή s στο μάθημα c
- ❑ Αν δεν υπάρχει, τότε το εσωτερικό NOT EXISTS ισχύει
- ❑ Και άρα, το εξωτερικό NOT EXISTS βρίσκει ένα τέτοιο μάθημα, και απορρίπτει τον φοιτητή.

Χρήση του διπλού NOT EXISTS

```
SELECT s.student_id, s.student_name
FROM students s
WHERE NOT EXISTS (
  SELECT c.course_id
  FROM courses c
  WHERE c.professor = 'Καθηγητής X'
  AND NOT EXISTS (
    SELECT e.course_id
    FROM enrollments e
    WHERE e.student_id = s.student_id
    AND e.course_id = c.course_id
  )
);
```

- ❑ Η Άννα είναι η μόνη που έχει εγγραφεί και στο 101 και στο 102, δηλαδή σε όλα τα μαθήματα του Καθηγητή X.

Αντί του διπλού NOT EXIST

```
SELECT e.student_id, s.student_name
FROM enrollments e, courses c, students s
WHERE e.course_id = c.course_id
      AND e.student_id = s.student_id
      AND c.professor = 'Καθηγητής X'
GROUP BY e.student_id, s.student_name
HAVING COUNT(DISTINCT c.course_id) = (
  SELECT COUNT(*)
  FROM courses
  WHERE professor = 'Καθηγητής X'
);
```

- ❑ **FROM** μεταξύ enrollments, courses και students για να φέρουμε όλα τα απαραίτητα πεδία!
- ❑ **WHERE**: κρατάμε μόνο τα μαθήματα του Καθηγητή X
- ❑ **GROUP BY**: ομαδοποιούμε ανά φοιτητή
- ❑ **HAVING**: κρατάμε μόνο όσους έχουν εγγραφεί σε **όλα** τα μαθήματα του Καθηγητή X (ίδιος αριθμός με τον συνολικό)

Αυτοσυσχέτιση

- ❑ Ένα κλασικό παράδειγμα όπου χρησιμοποιούμε τον ίδιο πίνακα δύο φορές με διαφορετικά ψευδώνυμα (aliases) είναι όταν θέλουμε να συγκρίνουμε γραμμές του πίνακα μεταξύ τους — δηλαδή να κάνουμε αυτοσυσχέτιση.
- ❑ Βρείτε τους φοιτητές που έχουν το ίδιο επίθετο με κάποιον άλλο φοιτητή.
 - ❑ Συγκρίνουμε κάθε φοιτητή με όλους τους άλλους, άρα χρησιμοποιούμε τον πίνακα `students` δύο φορές, π.χ. ως `s1` και `s2`.

```
SELECT DISTINCT s1.student_id, s1.first_name, s1.last_name
FROM students s1, students s2
WHERE s1.last_name = s2.last_name
AND s1.student_id <> s2.student_id;
```

student_id	first_name	last_name
1	Άννα	Παπαδοπούλου
2	Γιώργος	Νικολάου
3	Μαρία	Παπαδοπούλου
4	Νίκος	Παπαδόπουλος
5	Ελένη	Νικολάου
6	Κώστας	Βασιλείου

Αυτοσυσχέτιση

```
SELECT DISTINCT s1.student_id, s1.first_name, s1.last_name  
FROM students s1, students s2  
WHERE s1.last_name = s2.last_name  
AND s1.student_id <> s2.student_id;
```

- ☐ Συγκρίνει κάθε φοιτητή (s1) με κάθε άλλον (s2)
- ☐ Ελέγχει αν έχουν ίδιο επίθετο
- ☐ Φιλτράρει την περίπτωση όπου συγκρίνουμε τον φοιτητή με τον εαυτό του (s1.student_id <> s2.student_id)
- ☐ Χρήση DISTINCT για να μην εμφανιστούν διπλές εγγραφές

Αυτοσυσχέτιση

- ❑ Βρείτε τους φοιτητές που δεν έχουν κανέναν άλλο φοιτητή με το ίδιο επίθετο.

```
SELECT s1.student_id, s1.first_name, s1.last_name
FROM students s1
WHERE NOT EXISTS (
  SELECT s2.student_id
  FROM students s2
  WHERE s2.last_name = s1.last_name
  AND s2.student_id <> s1.student_id
);
```

student_id	first_name	last_name
1	Άννα	Παπαδοπούλου
2	Γιώργος	Νικολάου
3	Μαρία	Παπαδοπούλου
4	Νίκος	Παπαδόπουλος
5	Ελένη	Νικολάου
6	Κώστας	Βασιλείου

- ❑ Για κάθε φοιτητή s1, ελέγχει αν δεν υπάρχει άλλος φοιτητής s2 με το ίδιο επίθετο αλλά διαφορετικό student_id
- ❑ Αν δεν υπάρχει τέτοιος, τότε ο s1 έχει μοναδικό επίθετο και περνάει στο αποτέλεσμα

MySQL

General Info: <https://dev.mysql.com/>

Διανέμεται και υποστηρίζεται από την Oracle.

MySQL Server απομακρυσμένη σύνδεση

- ❑ Command – line client μέσω terminal:
 - ❑ `mysql -h edudb.di.uoa.gr -P 3306 -u singer -p`
 - ❑ όπου `edudb.di.uoa.gr` είναι η διεύθυνση της μηχανής (host server) που φιλοξενεί το mysql server (application)
- ❑ Μέσω του MySQL Workbench interface

MySQL βασικές εντολές

- ❑ Λίστα βασικών εντολών:
 - ❑ `mysql>HELP`
- ❑ Mysql server status
 - ❑ `mysql>STATUS`
- ❑ Έξοδος από client
 - ❑ `mysql>QUIT`

- ❑ Διαθέσιμες Βάσεις
 - ❑ `mysql>SHOW DATABASES;`
- ❑ Επιλογή Βάσης (π.χ., Βάση songs)
 - ❑ `mysql>USE songs;`
- ❑ Διαθέσιμοι πίνακες της Βάσης songs
 - ❑ `mysql>SHOW TABLES;`

MySQL βασικές εντολές

- ❑ Περιγραφή ενός πίνακα
 - ❑ `mysql> SHOW COLUMNS FROM table;`
 - ❑ `mysql> DESCRIBE SP;`
- ❑ Δείξε πώς δημιουργήθηκε ένας πίνακας
 - ❑ `mysql> SHOW CREATE TABLE SP;`

Songs DB

